

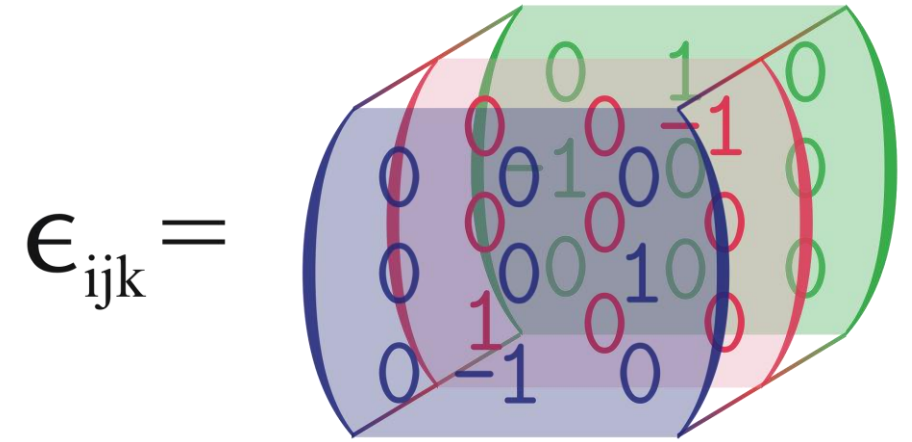


Numpy

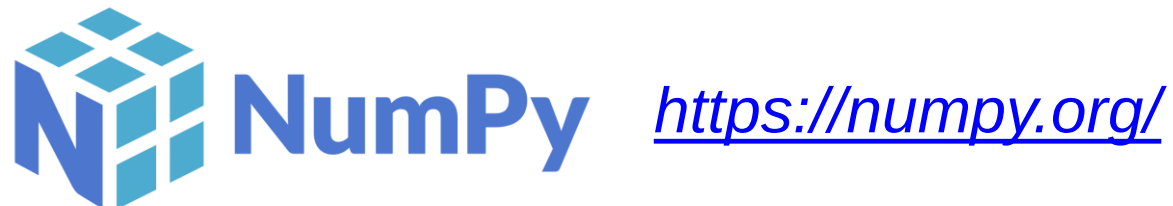
- What is it?
- Why NumPy?
- Array
- Random numbers
- Random numbers and reproducibility
- Random numbers and distributions
- Reshaping
- Indexing
- Iteration
- Operators
- Scalar Product
- Matrix-vector product
- Product Row-Columns
- Input/Output
- Cheat sheet

Numpy: What is it?

- **NumPy** is a Python library for *numerical computation on scalars, vectors, matrices*, and more generally *multidimensional tensors*.
- **NumPy** has advanced features for indexing and manipulating data of arbitrary size.
- It is efficient for linear algebra operations, optimization, sampling of random variables (statistics) ...
- It is *open-source* and **freely accessible** (BSD, free license).
- It is the basic tool, together with Pandas, for representing scientific information and datasets which will then be given as input to AI and ML tools.



Source: <https://it.wikipedia.org/wiki/Tensor>



Numpy: Why?

- Basic Python lists and data structures are convenient but **slow**.
- Operations performed on NumPy structures are up to 50 times faster than the same operations performed on Python data structures.
- NumPy arrays are stored as a single block in memory unlike Python lists, making their use more efficient.
- Arrays, matrices and tensors are in fact widely used in Data Science, where **speed** and **computational resources** are important.
- NumPy arrays are called **ndarrays** (=n-dimensional array).

Numpy Arrays

- NOTE:_1 In this section, *to report the strings returned from Numpy prints, the values will be separated by commas to improve the readability of the slide.*
However, as we will see on IDEs and interpreters, NumPy *does not divide values with commas*, so be careful!
 - NOTE_2: NumPy considers the "*mathematical comma*" with the dot character , e.g., **4.3**.
 - To use NumPy and create a basic array, you do the following:
 - `import numpy as np`
- ```
x = np.zeros(5)
print(x) # Out: [0., 0., 0., 0., 0.]
print(x.shape) # Out: (5,)
print(x.ndim) # Out: 1
print(x.dtype) # Out: float64
```
- <https://numpy.org/doc/stable/reference/generated/numpy.zeros.html#numpy.zeros>
  - <https://numpy.org/doc/stable/reference/generated/numpy.shape.html#numpy.shape>
  - <https://numpy.org/doc/stable/reference/generated/numpy.ndarray.ndim.html>
  - <https://numpy.org/doc/stable/reference/arrays.dtypes.html>

# Numpy Arrays: Declarations

- Initializing an array (vector, matrix or tensor) with default values (0 or 1) is quite common, e.g., for debugging purposes.

- ```
x = np.zeros(3)
print(x) # Out: [0., 0., 0.]
```

- ```
x = np.diag(np.ones(4))
print(x)
Out: [[1., 0., 0., 0.]
 [0., 1., 0., 0.]
 [0., 0., 1., 0.]
 [0., 0., 0., 1.]]
```

- ```
x = np.diag([4, 7, 1])
print(x)
# Out: [[4, 0, 0]
        [0, 7, 0]
        [0, 0, 1]]
```

- <https://numpy.org/doc/stable/reference/generated/numpy.diag.html#numpy.diag>
- <https://numpy.org/doc/stable/reference/generated/numpy.ones.html#numpy.ones>

Numpy Arrays: Declarations

- **arange** function creates a sequence of numeric values with *steps* chosen by the user.
 - **np.arange([start,] stop, [step,] ...)**
 - `x = np.arange(0.0, 1.0, 0.1)`
`print(x)` # Out: [0., 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9]
 - `x = np.arange(0.0, 1.1, 0.1)`
`print(x)` # Out: [0., 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.]
 - `x = np.diag(np.arange(3))`
`print(x)` # Out: [[0, 0, 0]
 [0, 1, 0]
 [0, 0, 2]]

<https://numpy.org/doc/stable/reference/generated/numpy.arange.html>

Numpy : Random Numbers



- NumPy allows to generate pseudo-random numbers, a feature that is useful in data science activities.
- A similar functionality is also offered by Python.

- **`np.random.method_to_call`**

- `x = np.random.uniform(0, 1)`
`print(x)` # Out: 0.1590282223151751 (*may vary*)
- `x = np.random.randint(5)`
`print(x)` # Out: 2 (*may vary*)
- `x = np.random.exponential(1.0)`
`print(x)` # Out: 2.9672208788264607 (*may vary*)

- <https://numpy.org/doc/stable/reference/random/index.html#module-numpy.random>
- <https://numpy.org/doc/stable/reference/random/legacy.html>
- <https://docs.python.org/3/library/random.html>

Numpy : Random Numbers and Reproducibility

- Developing and validating a data model or other problem for any application with (*pseudo*)**random numbers at each run** is **ineffective**.
- To obtain the **reproducibility of the results in the development and debugging phase**, the **concept of seed** was introduced.
- The `np.random.seed(seed_number)` function sets a seed based on an input value, which ensures that the results are always reproducible starting from the same seed value.
- NOTE: If you use **two libraries with two different random function implementations**, you will need *to set a seed for each library*.
 - `np.random.seed(0)` - NumPy
 - `random.seed(0)` – Python built-in
 - <https://docs.scipy.org/doc/numpy-1.15.1/reference/generated/numpy.random.seed.html>
 - <https://docs.python.org/3/library/random.html>

Numpy: Random Numbers and Distributions

- Numpy generators can be directly used to create arrays of random numbers following a certain rule or statistical distribution.
 - `x = np.random.exponential(1.0, size=(1, 5))`
`print(x)` # Out: `[[3.01012143 1.31674569 0.91294255 0.16962487 0.16959629]]`
 - `x = np.random.randint(1, 100, size=(2, 2))`
`print(x)` # Out: `[[84, 73]
[86, 67]]`
 - `x = np.random.uniform(1, 2, size=(3, 3))`
`print(x)` # Out: `[[1.20550941, 1.77908499, 1.41648758]
[1.92159824, 1.56921853, 1.70311]
[1.38104375, 1.05898555, 1.51192987]]`
 - <https://numpy.org/doc/1.16/reference/routines.random.html>

Numpy: Reshaping

- The size of Numpy arrays can be changed with the **reshape** and **ravel** functions .
- **Ravel** returns the array to a 1D vector.

```
• x = np.random.randint(1, 100, size=(1, 6))  
  y = x.reshape(2, 3)  
  z = y.ravel()  
  
print(x)      # Out:      [[35, 69, 80 , 52, 44, 9]]  
print(y)      # Out:      [[35, 69, 80]  
                        [52, 44, 9]]  
  
print(z)      # Out:      [35, 69, 80, 52, 44, 9]
```

- <https://numpy.org/doc/stable/reference/generated/numpy.reshape.html>
- <https://numpy.org/doc/stable/reference/generated/numpy.ravel.html>

Numpy: Indexing

- To index the elements of a multidimensional array you use square brackets, e.g.: **variable_name[x_position, y_position]**

- `x = np.random.randint(1, 100, size=(5, 5))`

- `print(x)` # Out:

```
[[28 30 46  4 94]
 [45 63  8 65 22]
 [74 37 78  3  8]
 [49 94 91 26 77]
 [48 82 51 37 34]]
```
- `print(x[2, :])` # Out:

```
[[28 30 46  4 94]
 [45 63  8 65 22]
 [74 37 78  3  8]
 [49 94 91 26 77]
 [48 82 51 37 34]]
```

- `print(x[2, 3])`
Out:

```
[[28 30 46  4 94]
 [45 63  8 65 22]
 [74 37 78  3  8]
 [49 94 91 26 77]
 [48 82 51 37 34]]
```
- `print(x[:, 3])`
Out:

```
[[28 30 46  4 94]
 [45 63  8 65 22]
 [74 37 78  3  8]
 [49 94 91 26 77]
 [48 82 51 37 34]]
```
- `print(x[:, 1:3])`
Out:

```
[[28 30 46  4 94]
 [45 63  8 65 22]
 [74 37 78  3  8]
 [49 94 91 26 77]
 [48 82 51 37 34]]
```

<https://numpy.org/doc/stable/user/basics.indexing.html>

Numpy : Iterations

- To iterate over the rows of a multidimensional array you can use a *for loop* combined with *slicing*:
- ```
x = np.random.randint(1, 100, size=(5, 5))
for row in x:
 print(row)
```

```
Out: [62, 9, 76, 34, 99]
 [10, 20, 6, 87, 85]
 [7, 52, 59, 57, 57]
 [93, 73, 94, 65, 97]
 [56, 62, 48, 58, 61]
```
- ```
x = np.random.randint(1, 100, size=(5, 5))  
for row in x:  
    print(row[2])
```

```
# Out:  15  
        6  
        59  
        94  
        48
```
- <https://numpy.org/doc/stable/reference/arrays.nditer.html>

Numpy: Operators

- Operations between arrays and scalars are extended to all elements of the array:

- `x = np.random.randint(1, 100, size=(1,5))`

- `print(x)` # Out: `[[52 93 15 72 61]]`

- `y = x*5`

- `print(y)` # Out: `[[260 465 75 360 305]]`

- `z = y**2`

- `print(z)` # Out: `[[ 67600 216225 5625 129600 93025]]`

- Note: This is a characteristic of algebraic operations. NumPy simply follows the mathematical rules and respects them accordingly.

Numpy: Operators

- It is also possible to apply operations to sub-arrays, leaving the rest of the multidimensional array unchanged:

- `x = np.random.randint(1, 100, size=(5,5))`

```
print(x)      # Out:      [[4, 4, 2, 5, 4]
                        [2, 5, 4, 7, 5]
                        [9, 9, 3, 7, 4]
                        [5, 9, 8, 7, 2]
                        [6, 3, 6, 4, 7]]
```

- `x[2:4, 2:4] = x[2:4, 2:4] * 10`

```
print(x)      # Out:      [[4, 4, 2, 5, 4]
                        [2, 5, 4, 7, 5]
                        [9, 9, 30, 70, 4]
                        [5, 9, 80, 70, 2]
                        [6, 3, 6, 4, 7]]
```

Numpy: Operators



- When performing operations between different arrays, if their sizes are incompatible an error is generated:
 - ```
x = np.random.randint(1, 10, size=(3,3))
y = np.random.randint(1, 10, size=(2,2))
z = x + y
Out: ValueError: operands could not be broadcasted together with
shapes (3,3) (2,2)
```
- This is a characteristic of algebraic operations. NumPy simply follows the mathematical rules and respects them accordingly.

# Numpy: Operators

- NumPy tries to extend operations in case of compatible sizes, with *implicit assumptions* :
  - `x = np.random.randint(1, 10, size=(3,3))`  
`print(x)` # Out: 

```
[[3 2 3]
 [9 7 6]
 [7 2 5]]
```
  - `y = np.random.randint(1, 10, size=(1,3))`  
`print(x)` # Out: 

```
[[4 7 2]]
```
  - `z = x + y`  
`print(z)` # Out: 

```
[[7 9 5]
 [13 14 8]
 [11 9 7]]
```
- In this case, NumPy adds the vector of size 3 row by row in our matrix having 3 columns.
- The implicit assumption is to consider the y matrix as a 3x3 matrix where the first row is repeated 3 times.
- To test: <https://matrixcalc.org/it/>

# Numpy: Scalar Product

- Among the various operations of linear algebra, there is the *scalar product* .
- It is calculated between two *vectors* (or *arrays* in NumPy), and is nothing more than **the sum of the multiplications between the elements of the two vectors in the same positions** .
  - `x = np.random.randint(1, 10, size=(10))`  
`print(x)` # Out: [8 5 4 5 7 3 7 2 2 5]
  - `y = np.random.randint(1, 10, size=(10))`  
`print(y)` #Out: [3 3 5 4 6 2 3 1 5 3]
  - `z = np.dot(x, y)`  
`print(z)` #Out: 175

`[8 5 4 5 7 3 7 2 2 5]`  
`[3 3 5 4 6 2 3 1 5 3]`

- Calculation:  $(8*3) + (5*3) + (4*5) + (5*4) + \dots$

# Numpy: Matrix-Vector Product

- Another algebraic operation is the *matrix-vector product* .

- `x = np.random.randint(1, 10, size=(5))`  
`print(x)` #Out: [8 5 4 7 3]
- `y = np.random.randint(1, 10, size=(5, 2))`  
`print(y)` #Out:   
[[2 6]  
[3 9]  
[4 4]  
[3 9]  
[2 7]]
- `z = np.dot(x, y)`  
`print(z)` #Out: [ 74 193]

$$\begin{matrix} \text{m righe} \downarrow \\ \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{pmatrix} \end{matrix} \cdot \begin{matrix} \xrightarrow{\text{n colonne}} \\ \begin{pmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \end{pmatrix} \end{matrix}$$

# Numpy: Rows-Columns Product

- Another algebraic operation is the *row-column product*.

- ```
x = np.random.randint(1, 10, size=(3, 5))
```



```
print(x) #Out:  [[1 3 7 6 3]
```

```
                [5 2 5 7 3]
```

```
                [6 2 8 5 6]]
```

- ```
y = np.random.randint(1, 10, size=(5, 2))
```

```
print(y) #Out: [[6 4]
```

```
 [5 6]
```

```
 [7 1]
```

```
 [5 3]
```

```
 [1 3]]
```

- ```
z = np.dot(x, y)
```



```
print(z) #Out:  [[103 56]
```

```
                [113 67]
```

```
                [133 77]]
```

- Other linear algebra operations offered by NumPy include:

- Eigenvalues, eigenvectors
- Solving linear systems
- ... And other things you can find in a Linear Algebra course! 🧐

$$\begin{matrix} \text{m righe} \downarrow & \begin{pmatrix} \boxed{a_{11} & a_{12}} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{pmatrix} & \cdot & \begin{pmatrix} \boxed{b_{11}} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \end{pmatrix} \end{matrix}$$

Numpy: Input/Output



- NumPy offers tools for interacting with files, i.e., reading information from files or writing information to files.
- Files can be written (i.e. saved) to files in ***binary*** or ***text format*** .
 - **Binary** : takes up less disk space, but if you open the file with another program (eg Notepad) it is not understandable;
 - **Textual** : vice versa.

Numpy: Input/Output

- ```
x = np.random.randint(1, 100, size=(5, 5))
np.savetxt("X1.txt", x)
np.savetxt("X2.txt", x, fmt='%1.5e')
np.savetxt("X3.txt", x, fmt='%d', delimiter=',')
```
- Format specifiers:
  - **C** : character (char)
  - **d** or **i** : signed integer
  - **e** or **E** : scientific notation with e (Neper number – 1.5 represents the precision of the number, in this case 1 decimal place after the decimal point)
  - **f** : floating point number
  - **g** , **G** : Use the shorter notation between e, E, f
  - **s** : string.

# Numpy: Input/Output

- ```
x = np.random.randint(1, 100, size=(5, 5))  
y = np.random.randint(1, 100, size=(5, 5))  
np.savez("data.npz", X=x, Y=y)  
data = np.load('data.npz')  
print(data['X'])  
print(data['Y'])
```
- Binary formats available:
 - **np.save** : Standard **.npy** format
 - **np.savez** : Uncompressed binary file, **.npz** format (for multiple arrays)
 - **np.savez_compressed** : Compressed version of **.savez** .
 - **np.load** : to load data in **.npy** or **.npz** format .

Numpy: Cheatsheet

[LINK](#)

Python For Data Science Cheat Sheet

NumPy Basics

Learn Python for Data Science Interactively at [www.DataCamp.com](https://www.datacamp.com)



NumPy

The **NumPy** library is the core library for scientific computing in Python. It provides a high-performance multidimensional array object, and tools for working with these arrays.

Use the following import convention:

```
>>> import numpy as np
```

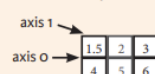


NumPy Arrays

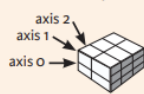
1D array



2D array



3D array



Creating Arrays

```
>>> a = np.array([1,2,3])
>>> b = np.array([(1.5,2,3), (4,5,6)], dtype = float)
>>> c = np.array([(1.5,2,3), (4,5,6)], [(3,2,1), (4,5,6)]], dtype = float)
```

Initial Placeholders

```
>>> np.zeros((3,4))
>>> np.ones((2,3,4),dtype=np.int16)
>>> d = np.arange(10,25,5)

>>> np.linspace(0,2,9)

>>> e = np.full((2,2),7)
>>> f = np.eye(2)
>>> np.random.random((2,2))
>>> np.empty((3,2))
```

Create an array of zeros
Create an array of ones
Create an array of evenly spaced values (step value)
Create an array of evenly spaced values (number of samples)
Create a constant array
Create a 2X2 identity matrix
Create an array with random values
Create an empty array

I/O

Saving & Loading On Disk

```
>>> np.save('my_array', a)
>>> np savez('array.npz', a, b)
>>> np.load('my_array.npy')
```

Saving & Loading Text Files

```
>>> np.loadtxt("myfile.txt")
>>> np.genfromtxt("my_file.csv", delimiter=',')
>>> np.savetxt("myarray.txt", a, delimiter=" ")
```

Data Types

```
>>> np.int64
>>> np.float32
>>> np.complex
>>> np.bool
>>> np.object
>>> np.string_
>>> np.unicode_
```

Signed 64-bit integer types
Standard double-precision floating point
Complex numbers represented by 128 floats
Boolean type storing TRUE and FALSE values
Python object type
Fixed-length string type
Fixed-length unicode type

Inspecting Your Array

```
>>> a.shape
>>> len(a)
>>> b.ndim
>>> e.size
>>> b.dtype
>>> b.dtype.name
>>> b. astype(int)
```

Array dimensions
Length of array
Number of array dimensions
Number of array elements
Data type of array elements
Name of data type
Convert an array to a different type

Asking For Help

```
>>> np.info(np.ndarray.dtype)
```

Array Mathematics

Arithmetic Operations

```
>>> g = a - b
array([[ 0.5,  0.,  0.],
       [-3., -3., -3.]])
>>> np.subtract(a,b)
>>> b + a
array([[ 2.5,  4.,  6.],
       [ 5.,  7.,  9.]])
>>> np.add(b,a)
>>> a / b
array([[ 0.66666667,  1.,  1.],
       [ 0.25,  0.4,  0.5]])
>>> np.divide(a,b)
>>> a * b
array([[ 1.5,  4.,  9.],
       [ 4.,  10.,  18.]])
>>> np.multiply(a,b)
>>> np.exp(b)
>>> np.sqrt(b)
>>> np.sin(a)
>>> np.cos(b)
>>> np.log(a)
>>> e.dot(f)
array([[ 7.,  7.]])
```

Subtraction
Subtraction
Addition
Addition
Division
Division
Multiplication
Multiplication
Exponentiation
Square root
Print sines of an array
Element-wise cosine
Element-wise natural logarithm
Dot product

Comparison

```
>>> a == b
array([[False,  True,  True],
       [False, False, False]], dtype=bool)
>>> a < 2
array([[True, False, False], dtype=bool)
>>> np.array_equal(a, b)
```

Element-wise comparison
Element-wise comparison
Array-wise comparison

Aggregate Functions

```
>>> a.sum()
>>> a.min()
>>> b.max(axis=0)
>>> b.cumsum(axis=1)
>>> a.mean()
>>> b.median()
>>> a.corrcoef()
>>> np.std(b)
```

Array-wise sum
Array-wise minimum value
Maximum value of an array row
Cumulative sum of the elements
Mean
Median
Correlation coefficient
Standard deviation

Copying Arrays

```
>>> h = a.view()
>>> np.copy(a)
>>> h = a.copy()
```

Create a view of the array with the same data
Create a copy of the array
Create a deep copy of the array

Sorting Arrays

```
>>> a.sort()
>>> c.sort(axis=0)
```

Sort an array
Sort the elements of an array's axis

Subsetting, Slicing, Indexing

Also see Lists

Subsetting

```
>>> a[2]
3
>>> b[1,2]
6.0
```

Select the element at the 2nd index
Select the element at row 1 column 2 (equivalent to b[1][2])

Slicing

```
>>> a[0:2]
array([1, 2])
>>> b[0:2,1]
array([ 2.,  5.])
>>> b[:1]
array([[1.5, 2., 3.]])
>>> c[1,...]
array([[ 3.,  2.,  1.],
       [ 4.,  5.,  6.]])
```

Select items at index 0 and 1
Select items at rows 0 and 1 in column 1
Select all items at row 0 (equivalent to b[0:1, :])
Same as [1, :, :]
Reversed array a

Boolean Indexing

```
>>> a[a<2]
array([1])
>>> b[[1, 0, 1, 0], [0, 1, 2, 0]]
array([ 4.,  2.,  6.,  1.5])
>>> b[[1, 0, 1, 0]][:, [0,1,2,0]]
array([[ 4.,  5.,  6.,  4.],
       [ 1.5,  2.,  3.,  1.5],
       [ 4.,  5.,  6.,  4.],
       [ 1.5,  2.,  3.,  1.5]])
```

Select elements from a less than 2
Select elements (1,0), (0,1), (1,2) and (0,0)
Select a subset of the matrix's rows and columns

Array Manipulation

Transposing Array

```
>>> i = np.transpose(b)
>>> i.T
```

Permute array dimensions
Permute array dimensions

Changing Array Shape

```
>>> b.ravel()
>>> g.reshape(3,-2)
```

Flatten the array
Reshape, but don't change data

Adding/Removing Elements

```
>>> h.resize((2,6))
>>> np.append(h,g)
>>> np.insert(a, 1, 5)
>>> np.delete(a,[1])
```

Return a new array with shape (2,6)
Append items to an array
Insert items in an array
Delete items from an array

Combining Arrays

```
>>> np.concatenate((a,d),axis=0)
array([ 1,  2,  3, 10, 15, 20])
>>> np.vstack((a,b))
array([[ 1.,  2.,  3.],
       [ 1.5,  2.,  3.],
       [ 4.,  5.,  6.]])
>>> np.r_[e,f]
array([[ 7.,  7.,  1.,  0.],
       [ 7.,  7.,  0.,  1.]])
>>> np.column_stack((a,d))
array([[ 1, 10],
       [ 2, 15],
       [ 3, 20]])
>>> np.c_[a,d]
```

Concatenate arrays
Stack arrays vertically (row-wise)
Stack arrays vertically (row-wise)
Stack arrays horizontally (column-wise)
Create stacked column-wise arrays
Create stacked column-wise arrays

Splitting Arrays

```
>>> np.hsplit(a,3)
[array([1]),array([2]),array([3])]
>>> np.vsplit(c,2)
[array([[ 1.5,  2.,  1.],
       [ 4.,  5.,  6.] ]]),
 array([[ 3.,  2.,  3.],
       [ 4.,  5.,  6.] ])]
```

Split the array horizontally at the 3rd index
Split the array vertically at the 2nd index

DataCamp
Learn Python for Data Science Interactively

